Security rules & indexes

Epic: EPIC-017 (#148) · **Requirements:** A-05, A-08 · **Sources:** firestore.rules, firestore.indexes.json, schema §7 index plan and §9 security, rules tests

> **Status:** merged and deployed: field validation (#153), index review (#156).

Problem

Kavindu's money syncs to the cloud (**A-05**), so the server must protect it from other users and from malformed writes. That covers a buggy app version, a stale offline client, or a hand-crafted request. Every record is meant to be structured (**A-08**), so the server should enforce that structure rather than trust the client. Indexes must match the queries the app actually runs, no more.

Design decision — rules

- **Owner-only.** Every read and write under users/{uid} requires request.auth.uid == uid. There are no collection-group reads and no shared data.
- **Every collection is validated field by field**: profile, accounts, categories, transactions, invoices, recurring and goals.
  - hasOnly([...]) rejects unknown fields.
  - Required fields are type-checked (reqStr, reqInt, reqBool, reqTs). Optionals may be absent or null (optStr, …), because the app's maps write nulls for unset optionals.
  - Enums are checked against their allowed names (inSet).
- **Business invariants on the server**, mirroring the domain validation:
  - money is a positive integer, except crypto income, which may be negative;
  - an expense needs a category and a spend kind;
  - a transfer needs a different destination account and must be ACTUAL;
  - currencies are LKR, USD or USDT;
  - a recurring day of month is 1–28.
- **Write-once fields.** createdAt never changes on update (keepsCreatedAt), and neither does a goal's startedAt. This is also what stops a second device re-creating a document under an existing id (e.g. a recurring confirm for the same month).
- **No hard deletes.** allow delete: if false. Deleting is a soft delete (deletedAt), which keeps undo and an audit trail. The sync mirror removes soft-deleted rows from Room.

Design decision — indexes

The app reads from **Room**, never Firestore, for screens (ADR-0004). Firestore is queried only by the sync listeners, one per collection, each **unfiltered** (no where, no orderBy). Month windows, sums and ordering are Room queries. So:

- **No composite indexes.** Firestore needs one for any compound query its automatic single-field indexes can't serve: a filter on one field ordered by another, inequalities on more than one field, most or queries, and so on. This app's query inventory (§7.4) is entirely unfiltered listeners, so none of those cases arises. Unused composites cost write latency and storage.
- **The automatic single-field indexes** serve the unfiltered listeners.
- **Free-text fields are exempted** from single-field indexing, since they are never queried: transactions.note, invoices.clientName, invoices.projectName, invoices.milestone and invoices.note.

- **Windowed listener.** If a listener were ever windowed (`occurredAt >= cutoff`), it filters on one field and uses the automatic single-field index, so it still needs no composite.

The full query inventory, with each query, its filter, whether it needs an index and why, is in schema §7.4. That table is the answer to "explain your indexing strategy".

Data flow

```
App write — Repository — RemoteDataSource — toMap() → Firestore → firestore.rules (owner + validation) → stored
                                                                                  └ rejected: local cache rolls back; listener restores Room
Sync — SyncManager — unfiltered snapshot listener per collection → automatic single-field index → Room
```

Edge cases handled

Case	Behaviour
Another user's uid in the path	Denied for reads and writes.
An unknown or misspelled field	Denied (`hasOnly`).
A string where a number belongs, or a float amount	Denied (`reqInt`).
A negative expense	Denied; only crypto income may be negative.
A transfer to the same account	Denied.
Rewriting `createdAt` or a goal's `startedAt`	Denied.
A hard delete	Denied; use a soft delete.
A malformed write while offline	Accepted locally, rejected by the server on sync; the cache rolls back and the listener restores Room.

Screenshots

TODO(M4) img/rules-rejected-write.png — Rules Playground rejecting a malformed transaction
A malformed transaction (a string amount) rejected by the deployed rules: the Firebase Console Rules Playground, or the emulator test run in rules.md.

Tests

- `rules-tests/`: emulator tests for each collection's accept and reject cases (see Docs/testing/rules.md).
- Index state: `firebase firestore:indexes` matches `firestore.indexes.json` after `firebase deploy --only firestore:indexes` (#156).